AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

FACULTY DEVELOPMENT PROGRAMME · ONE DAY

Agentic AI, Hands On

You will build six working agents today. The talking parts total about half an hour.

Dr. Abhijith Anandakrishnan · Assistant Professor

ASAI-FDP-01 · One-day workshop 2026-27

You have all used a chatbot.

Today you build the thing that uses the chatbot.

WHERE WE ARE GOING

The day

09:30 What an agent is · 30 min

10:15 Block 0 — setup

10:30 Block 1 — **Tool Use**

11:30 Block 2 — **Chaining and Routing**

12:15 Block 3 — **Reflection**

13:00 Lunch

14:00 Block 4 — **Retrieval**

15:00 Block 5 — **Multi-Agent capstone**

16:15 Where to take it next

FOUR AND A HALF HOURS OF THAT IS YOU, TYPING.

PART 1

A model does one thing.

Messages in. One message out. That is the entire interface.

THE WHOLE API

This is a language model

```
reply = model.invoke([
    SystemMessage("You are a maths tutor."),
    HumanMessage("What is 17 x 24?"),
])

reply.content   # "17 x 24 = 408"
```

Not a string in and a string out. A **list of messages** in.

Everything today is a way of deciding what goes into that list.

THE DISTINCTION THAT MATTERS

Model versus agent

A model

- Answers from what it absorbed in training
- Cannot look anything up
- Cannot act on the world
- One question, one answer, done
- Wrong confidently, and you cannot tell

An agent

- A model **plus tools**, in a **loop**
- Decides *for itself* when to use them
- Reads the result and decides again
- Runs until the goal is met
- You can see every step it took

Agent = model + tools + a loop + a stopping condition

Four things. Everything else today is a variation.

WHY THIS IS NOT JUST AN API CALL

The model chooses the sequence

You ask:

> How many credits are 23AID304 and 24AIM332 together?

You wrote **no** if-statements.

It decides to:

1. look up `23AID304`
2. look up `24AIM332`
3. add them with a calculator
4. answer

Change the question and the sequence changes.

That is the whole idea.

You describe capabilities. It works out the procedure.

PART 2

WHAT YOU BUILD TODAY

The catalogue we will use

1 · **Tool Use** Let it call functions.

2 · **Prompt Chaining** One call’s output feeds the next.

3 · **Routing** Classify first, then dispatch.

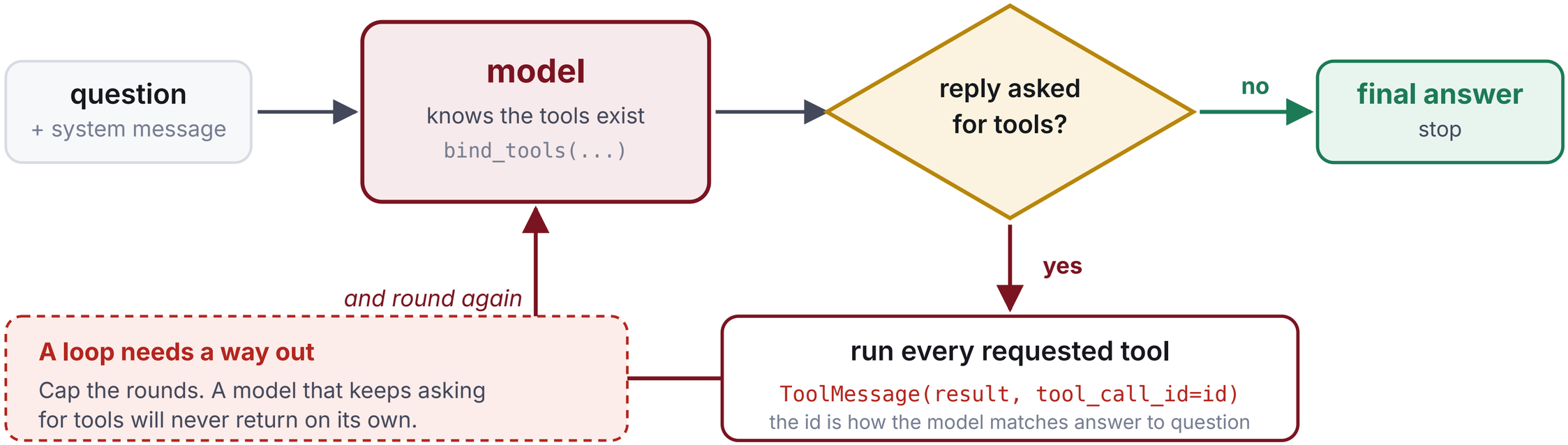
4 · **Reflection** It critiques its own draft.

5 · **Retrieval** Ground it in your documents.

6 · **Multi-Agent** A supervisor directing workers.

BLOCK 1

Tool Use: the loop is smaller than you expect



Two mistakes, both universal

Forgetting the tool_call_id

The model matches results to requests by id. Omit it and it reads an answer to a question it never asked.

Never stopping

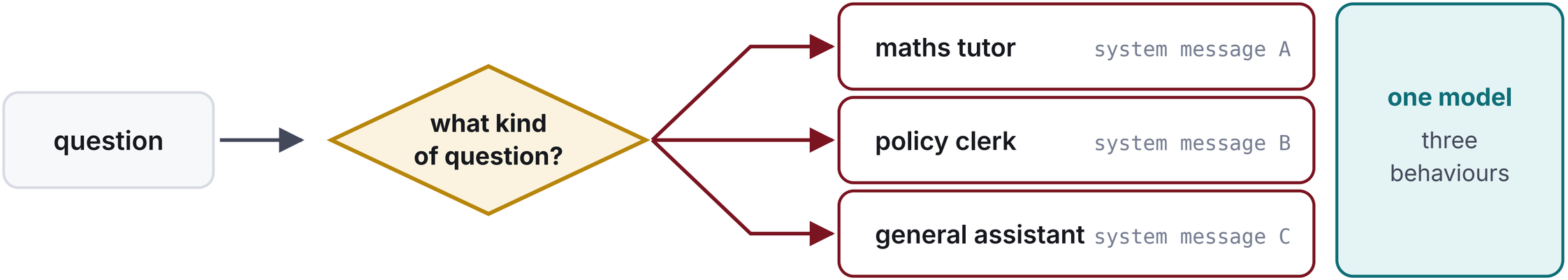
If your loop always runs a tool, it never returns. Every agent needs a step budget.

Chaining and Routing

CHAINING each call does one job, and you can see the middle

stage two reads stage one's output — not the original text

ROUTING classify first, then hand to a handler built for that kind



One instruction per call

Chaining

“Read this and give me a headline” is two jobs at once, done worse.

Split them, and you can **inspect the middle**.

Routing

Three handlers that differ **only by system message**.

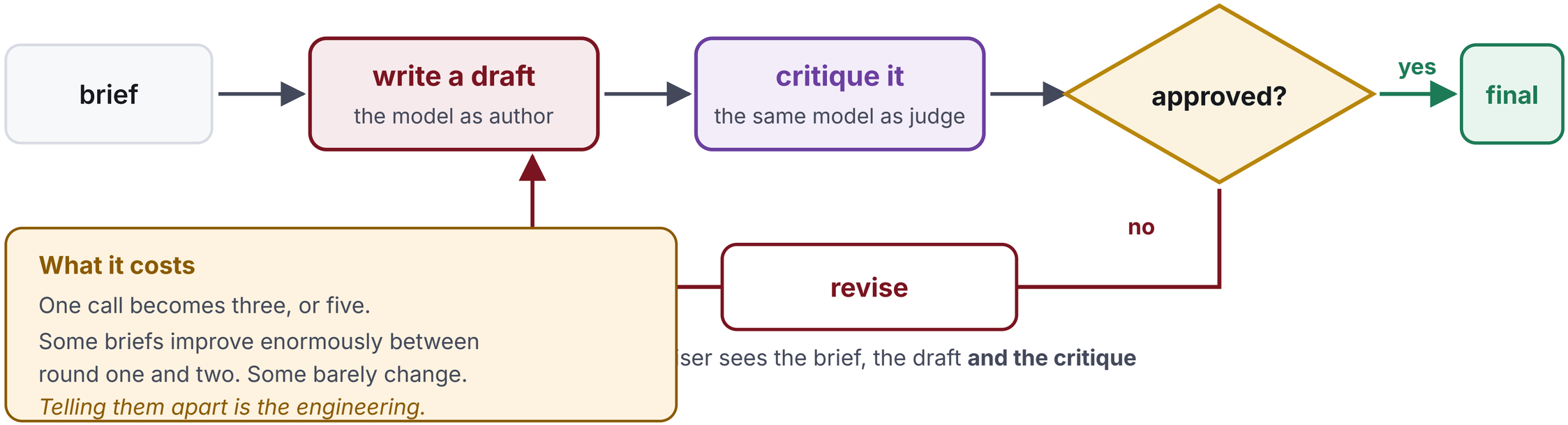
One model. Three behaviours. Nearly free.

Your router will get back "Maths."

Not “maths”. A router that does ROUTES[reply] crashes in front of the room.

BLOCK 3

Reflection: the model marks its own work



BLOCK 3 · THE OBVIOUS OBJECTION

"If it could write it better, why did it not?"

Because **writing** and **judging** are different tasks.

Given a finished text and asked what is wrong with it, the model attends to different things than while generating it.

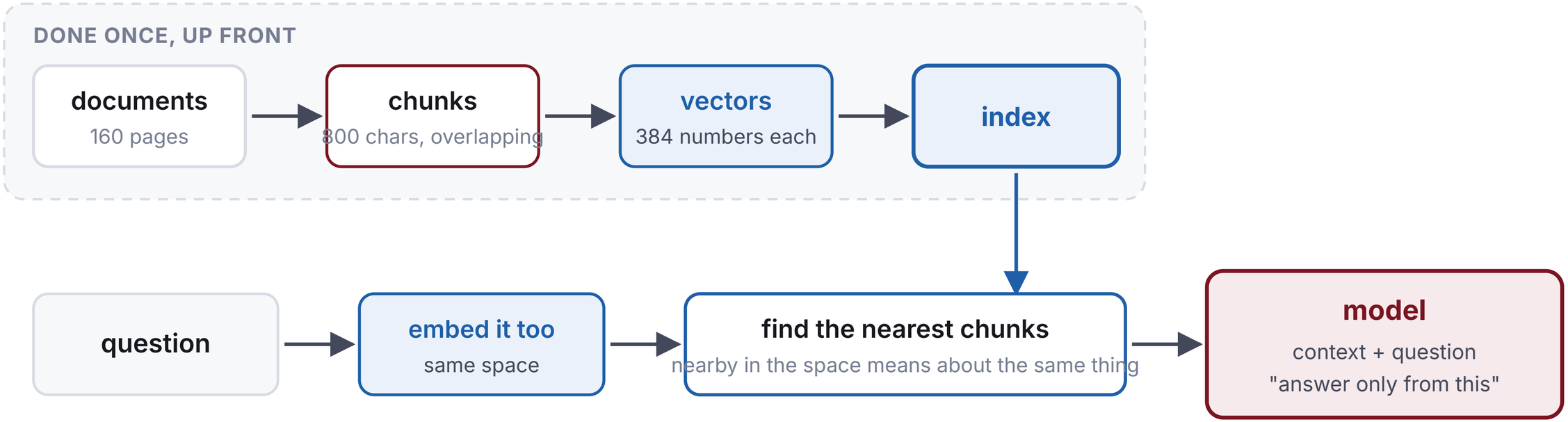
Reflection turns one call into three or five.

Sometimes that buys a much better answer. Sometimes it buys nothing.

Knowing which is the engineering.

BLOCK 4

Retrieval: do not give it the document



The bug you will not notice

Retrieve four perfect passages, then send the model the bare question. It answers from training data, sounds completely confident, and nothing in the output tells you the retrieval was thrown away.

Retrieving, then ignoring it

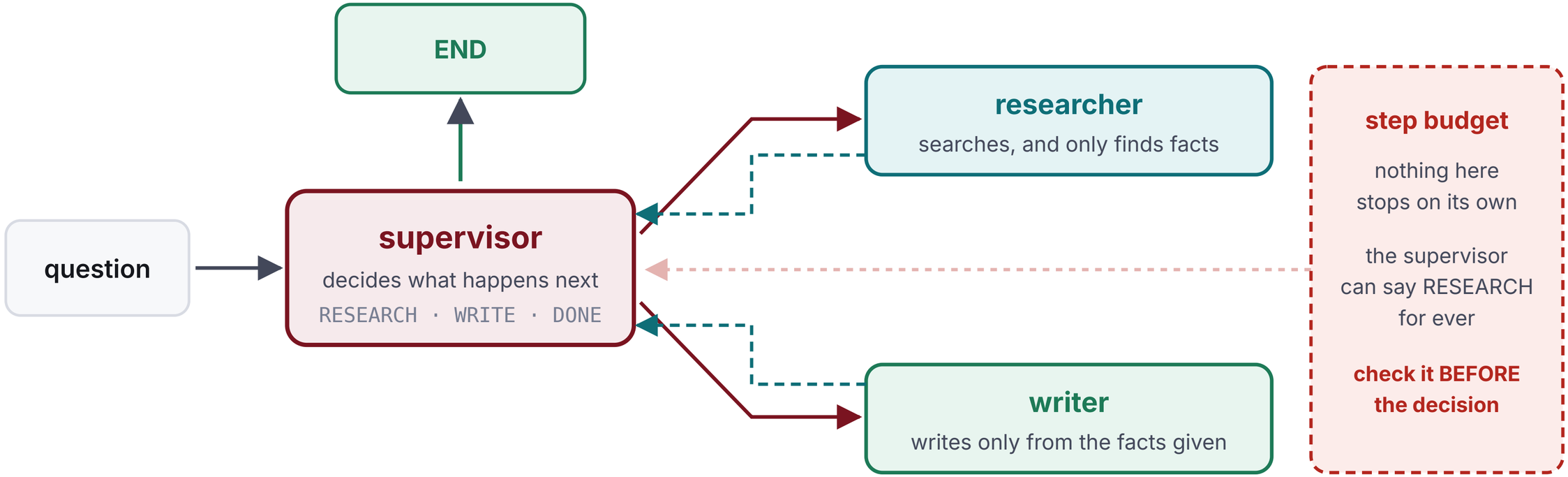
You build the index. You retrieve four perfect passages.

You send the model the bare question.

It answers from training data, sounds completely confident, and **nothing in the output tells you** the retrieval was ignored.

THE COMMONEST RAG BUG, BY A DISTANCE.

Multi-Agent: a supervisor, not a pipeline



a supervised team, not a fixed pipeline: the route differs per question

Nothing here stops on its own.

The supervisor can keep saying RESEARCH. Nothing raises. Nothing looks broken. It just runs.

Check the budget first

```
def route(state):
    if state["steps"] >= MAX_STEPS:
        return END          # first
    if state["next"] == "RESEARCH":
        return "research"
    ...
```

Checking the decision first grants **one extra round, every time**.

That is the sort of bug that shows up on the invoice rather than in the tests.

PART 3

THE SETUP

Everything runs on our own hardware

The model

`gemma-4-26B` on the School's server, two RTX 6000 Ada cards.

No account, no API key, no cost per call.

Your laptop

```
git clone <repo>
uv sync
uv run python -m agentic_fdp.check
```

You must be on campus wifi.

AN UNUSUAL CHOICE, DELIBERATELY

The tests do not call the real model

If they did

- The same solution passes and fails at random
- A wrong answer sometimes passes
- Nothing works without the network
- You cannot tell a bug from a bad day

What we do instead

- Tests run against a **scripted** model
- A failure means one thing: wrong wiring
- Works offline, and after today
- `demo.py` shows you the real behaviour

CHECKING YOUR WORK

Two ways, same answer

On your laptop

```
./selfcheck ex01
./selfcheck
```

Instant. Prints the reason it failed.

On GitHub

```
git commit -am "ex01"
git push
```

The Actions tab shows a block-by-block table.

Fill in the TODOs.

The scaffolding is written. You are wiring up the pattern, not fighting Python.

GROUND RULES

For the next six hours

Raise your hand early. Ten minutes stuck is nine minutes wasted; that is what today is for.

Run the demos. The tests prove it is wired right. The demos show you what it does.

Break things on purpose. Make a docstring vague. Halve the chunk size. Drop the budget to 2.

The pattern takes ten minutes to learn. Judgement takes experiments.

Block 0. Fifteen minutes.

Clone, sync, and get a reply out of the server.

THE END OF THE DAY

AFTER TODAY

It is yours to run

In your own teaching

Everything is public and portable. Point it at any OpenAI-compatible server with four environment variables.

The handbook has the timing plan and the notes for each block.

Further reading

Antonio Gulli, *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*

Twenty-one patterns. Each block cites its chapter.

The patterns are simple.

Knowing which one a problem needs is the part that takes practice. That starts now.